



RV College of
Engineering®

Go, change the world

How inconsistent Python/library versions can change the behavior of the ML model and what production-readiness steps to follow to ensure environment consistency

Alroy 1RV23AI012
Aniket 1RV23AI017
Anjali 1RV23AI018
Apoorva 1RV23AI020

Department of AIML



Behavior changes induced

a. Differences in numerical computation

- Libraries like NumPy, pandas, PyTorch, TensorFlow, and Scikit-Learn evolve.
- A minor version change can:
 - Alter default initialization,
 - Change random number generation,
 - Modify model handling internals (e.g., solver behavior, rounding, float precision),
 - Change preprocessing logic.

Result: The same input produces different predictions on Python 3.8 + scikit-learn 1.0 vs Python 3.10 + scikit-learn 1.4.



RV College of
Engineering®

Go, change the world

b. Serialization / deserialization differences

A model trained in one version of sklearn may not load identically another version due to:

- Changes in class definitions,
- Deprecation of certain attributes,
- Internal parameter restructuring.

This leads to behavior drift, even without retraining.



c. Preprocessing inconsistencies

Library version changes might alter:

- How categorical encoders handle unknown categories,
- How missing values are imputed,
- How default parameters behave (example:
OneHotEncoder(handle_unknown="ignore") was not default in older versions).

This breaks pipeline compatibility between environments.



Production-readiness steps to ensure environment consistency (MLOps best practices)

A. Use Environment Freezing

Guarantee that dev, staging, and production run the exact same packages.

Approaches:

1. requirements.txt with pinned versions
2. numpy==1.23.1scikit-learn==1.2.0python==3.8.16
3. Conda environment YAML
4. Poetry or Pipenv lock files

Never use floating versions like scikit-learn>=1.0.



B. Use Containerization (Docker)

The gold standard for production consistency.

- Package the model, code, and dependencies in a Docker image.
- Use the same image for development → staging → production.
- Tag images properly:
-
- medinsure-claim-model:v1.0.3

This prevents "works on my machine" problems.



RV College of
Engineering®

Go, change the world

Tools for reproducible training:

- MLflow
- DVC
- Weights & Biases
- Kubeflow Pipelines



D. Use CI/CD for ML (MLOps Pipelines)

A CI pipeline should:

- Build container
- Run unit tests on preprocessing and inference
- Check dependency conflicts
- Validate model performance
- Run security scans

A CD pipeline should:

- Promote the same artifact from dev → staging → production
- Block deployment if environment diff is detected



RV College of
Engineering®

Go, change the world

Continuously monitor for:

- New package releases
- Deprecated APIs
- Differences between environments



Conclusion

- Inconsistent Python and ML library versions can change the numerical behavior, preprocessing logic, and serialization format of the model, leading to different predictions in staging vs development.
- To ensure production-readiness, the team must freeze dependencies, use containers (e.g., Docker), record full training environments through MLflow/DVC, implement CI/CD pipelines that validate environment consistency, tag and version all model artifacts, and promote the same container image across dev → staging → production.
- This eliminates runtime mismatches and ensures reproducible, stable model behavior.